

Project 3: Supervised Classification on German Credit Risk (version 2)

Introduction

We once again approach the German Credit Risk data set. This time we will use a Multi-Layer Perceptron (MLP) neural network with Keras.

Data Preparation

Our data cleaning and preprocessing is as follows:

1. Drop the following categories: other_parties, foreign_worker, other_payment_plans
2. Outlier removal utilization IsolationForest
3. Create multiple pipelines and place them into a preprocessing pipeline. These pipelines handle specific functions. Imputing, Scaling, Encoding, and other functions are handled in their respective pipelines.

```
drop_columns = ["other_parties", "foreign_worker", "other_payment_plans"]
credit_data = credit_data.drop(columns=drop_columns)
```

```
X = credit_data.select_dtypes(include=[np.number])
isolation_forest = IsolationForest(contamination=0.12, random_state=42)
outlier_pred = isolation_forest.fit_predict(X)
```

```

def ratio_pipeline():
    return make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio, feature_names_out=ratio_name),
        StandardScaler()
    )

log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log1p, feature_names_out="one-to-one"), # Using log1p to handle zero values
    StandardScaler()
)

default_num_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler()
)

cat_pipeline = make_pipeline(
    OrdinalEncoder(),
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(drop='first')
)

preprocessing = ColumnTransformer(
    transformers=[
        ("num", default_num_pipeline, make_column_selector(dtype_include=np.number)),
        ("cat", cat_pipeline, make_column_selector(dtype_include=object)),
        ("log", log_pipeline, ["credit_amount", "duration", "age", "existing_credits", "num_dependents"]),
        #
        ("credit_amount_per_existing_credits", ratio_pipeline(), ["credit_amount", "existing_credits"]),
        ("installment_commitment_per_existing_credits", ratio_pipeline(), ["installment_commitment", "existing_credits"]),
        ("duration_per_credit_amount", ratio_pipeline(), ["duration", "credit_amount"]),
        ("age_per_duration", ratio_pipeline(), ["age", "duration"]),
        ("credits_per_duration", ratio_pipeline(), ["existing_credits", "duration"]),
        ("installments_per_duration", ratio_pipeline(), ["installment_commitment", "duration"]),
        ("age_per_credit_amount", ratio_pipeline(), ["age", "credit_amount"]),
        ("credits_per_residence", ratio_pipeline(), ["existing_credits", "residence_since"])
    ],
    remainder=default_num_pipeline
)

```

Model Building

We opted to go with 1 hidden layer. We were experiencing an overfitting issue which the additional layer amplified. We have set the layer sizes, batch normalization, and drop out rate. We can see trainable params and non-trainable params.

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 100)	5,700
batch_normalization (BatchNormalization)	(None, 100)	400
dropout (Dropout)	(None, 100)	0
dense_1 (Dense)	(None, 50)	5,050
batch_normalization_1 (BatchNormalization)	(None, 50)	200
dropout_1 (Dropout)	(None, 50)	0
dense_2 (Dense)	(None, 1)	51

Total params: 11,401 (44.54 KB)

Trainable params: 11,101 (43.36 KB)

Non-trainable params: 300 (1.17 KB)

Model Training

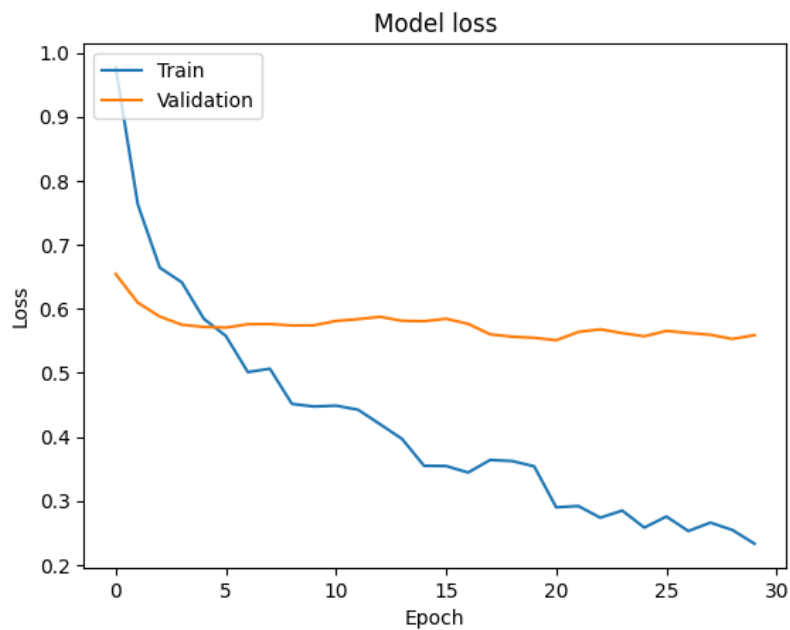
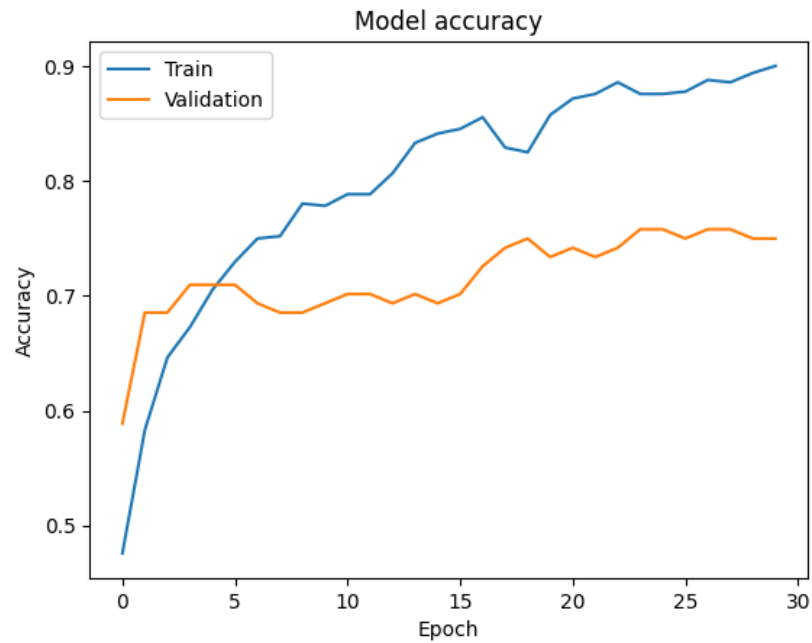
Our first training is a simple call with the selected parameters.

```
history = model.fit(credit_data_prepared, y_train, epochs=30, batch_size=32, validation_split=0.2)
```

✓ 3.4s

Model Evaluation

The following images show training vs validation for accuracy and loss. The blue line is our training which does quite well. The orange shows our validation which does not perform nearly as well. This is likely due to overfitting.



Here we define our TensorBoard parameters. We have to set file paths as well as specified certain documents. Note, we were operating in VS Code on Windows which required us to have to switch our slash path for Windows terminal.

```
import datetime
import os
import pathlib
import shutil

if os.path.exists("logs"):
    shutil.rmtree("logs")

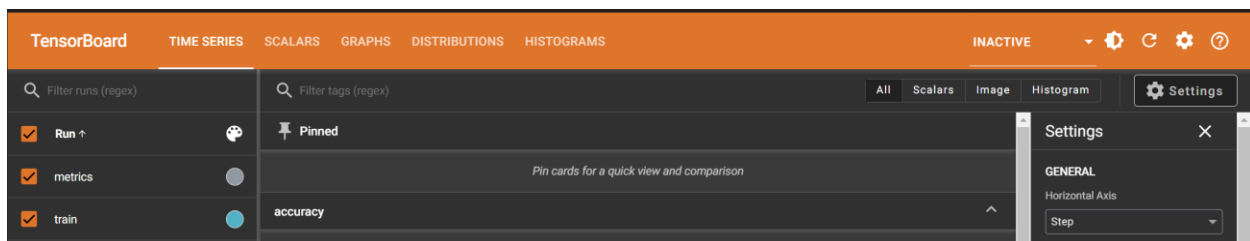
log_dir = pathlib.Path("logs") / "fit" / datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
log_dir.mkdir(parents=True, exist_ok=True)
log_dir_str = str(log_dir).replace("\\", "/")

tensorboard_callback = tf.keras.callbacks.TensorBoard(
    log_dir=log_dir_str,
    histogram_freq=1,
    write_graph=True,
    update_freq='epoch'
)

class DetailedTensorBoardCallback(tf.keras.callbacks.Callback):
    def __init__(self, log_dir):
        super(DetailedTensorBoardCallback, self).__init__()
        self.file_writer = tf.summary.create_file_writer(str(log_dir / 'metrics').replace("\\", "/"))

    def on_epoch_end(self, epoch, logs=None):
        with self.file_writer.as_default():
            for name, value in logs.items():
                tf.summary.scalar(name, value, step=epoch)
            self.file_writer.flush()

detailed_tb_callback = DetailedTensorBoardCallback(log_dir)
```



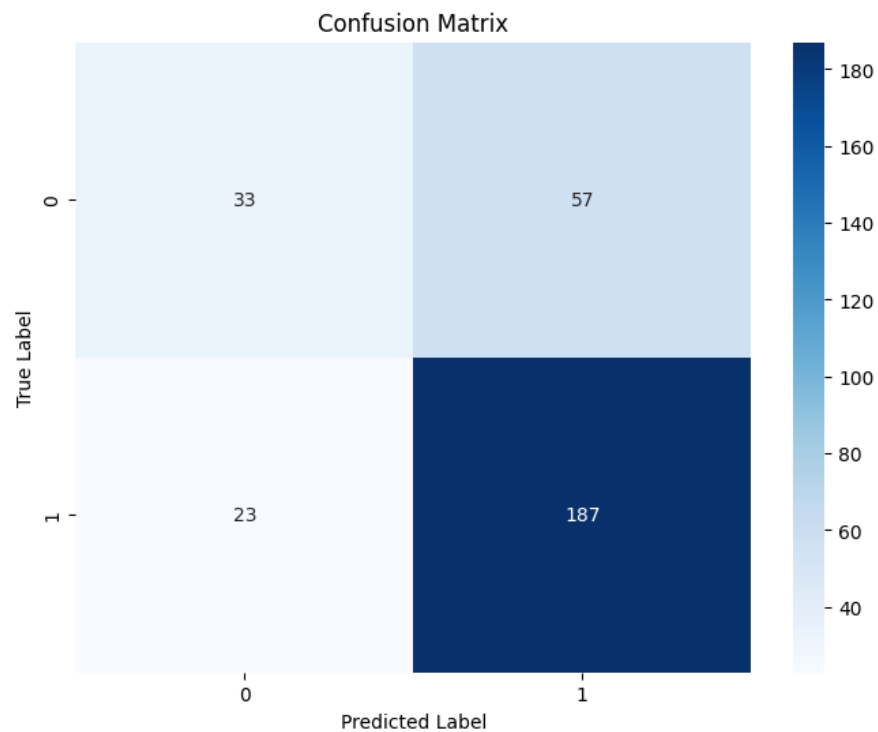
These are our TensorBoard charts. Not the similar shapes to our previous images. One big difference is that TensorBoard allows us to explore the information in more depth.



Prediction

Here are the first round of our predictions. Our overall accuracy is a modest 73%. This is a fair performance but we will try to do better with fine-tuning. There are a rather large number of false positives which is likely due to the imbalanced data which is heavier of “good credit” values as opposed to “bad credit” values.

```
Model Performance Metrics:  
Accuracy: 0.7333  
Precision: 0.7664  
Recall: 0.8905  
F1 Score: 0.8238  
ROC AUC Score: 0.7383
```



Hyperparameter Tuning

We have created a tuner model utilizing Keras Tuner as kt. Once again we opted for a single hidden layer for consistency.

```
class CreditModelTuner(kt.HyperModel):
    def build(self, hp):
        model = tf.keras.Sequential()

        # Tune input layer
        units_input = hp.Int('units_input', min_value=32, max_value=256, step=32)
        dropout_input = hp.Float('dropout_input', min_value=0.1, max_value=0.5, step=0.1)

        model.add(tf.keras.layers.Dense(
            units_input,
            activation='relu',
            input_shape=(credit_data_prepared.shape[1],)
        ))
        model.add(tf.keras.layers.BatchNormalization())
        model.add(tf.keras.layers.Dropout(dropout_input))

        # Tune hidden layer
        units_hidden = hp.Int('units_hidden', min_value=16, max_value=128, step=16)
        dropout_hidden = hp.Float('dropout_hidden', min_value=0.1, max_value=0.5, step=0.1)

        model.add(tf.keras.layers.Dense(
            units_hidden,
            activation='relu'
        ))
        model.add(tf.keras.layers.BatchNormalization())
        model.add(tf.keras.layers.Dropout(dropout_hidden))

        # Output layer
        model.add(tf.keras.layers.Dense(1, activation='sigmoid'))

        # Tune learning rate
        learning_rate = hp.Float('learning_rate', min_value=1e-4, max_value=1e-2, sampling='log')

        model.compile(
            optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
            loss='binary_crossentropy',
            metrics=['accuracy']
        )

        return model

    def fit(self, hp, model, *args, **kwargs):
```


We tune with Bayesian Optimization with our objective of val_accuracy.

```
tuner = kt.BayesianOptimization(  
    CreditModelTuner(),  
    objective='val_accuracy',  
    max_trials=30,  
    directory='credit_tuning',  
    project_name='german_credit'  
)
```

We establish call backs for TensorBoard that also utilizes Early Stopping.

```
tuning_callbacks = [  
    TensorBoard(  
        log_dir=str(tuning_log_dir / "tensorboard"),  
        histogram_freq=1  
    ),  
    EarlyStopping(  
        monitor='val_loss',  
        patience=5,  
        restore_best_weights=True  
    )  
]
```

We use tuner.search() on our prepared data to perform the hypertuning process.

```
print("\nStarting hyperparameter tuning...")  
tuner.search(  
    credit_data_prepared,  
    y_train,  
    epochs=30,  
    validation_split=0.2,  
    callbacks=tuning_callbacks  
)
```

We utilize the best parameters and create our model. We then train our model and create our history.

```
best_model = tuner.hypermodel.build(best_hps)

history1 = best_model.fit(
    credit_data_prepared,
    y_train,
    epochs=20,
    validation_split=0.2,
    callbacks=tuning_callbacks
)

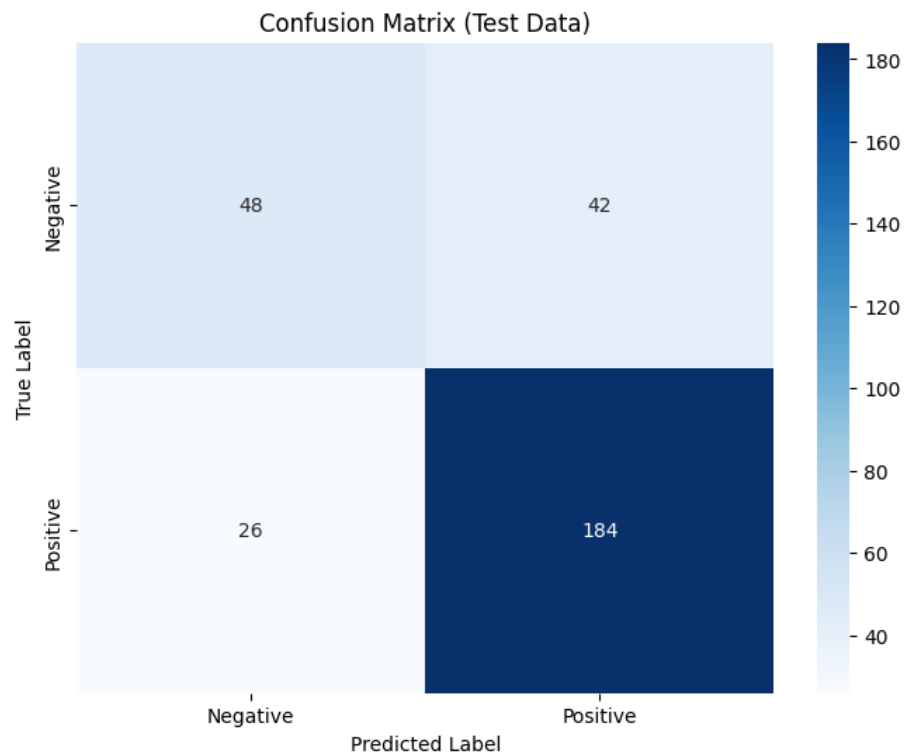
history = {
    'accuracy': history1.history['accuracy'],
    'val_accuracy': history1.history['val_accuracy'],
    'loss': history1.history['loss'],
    'val_loss': history1.history['val_loss']
}
```

Here are our final prediction stats. There is marked improvement from before. We finished with an accuracy of 79%.

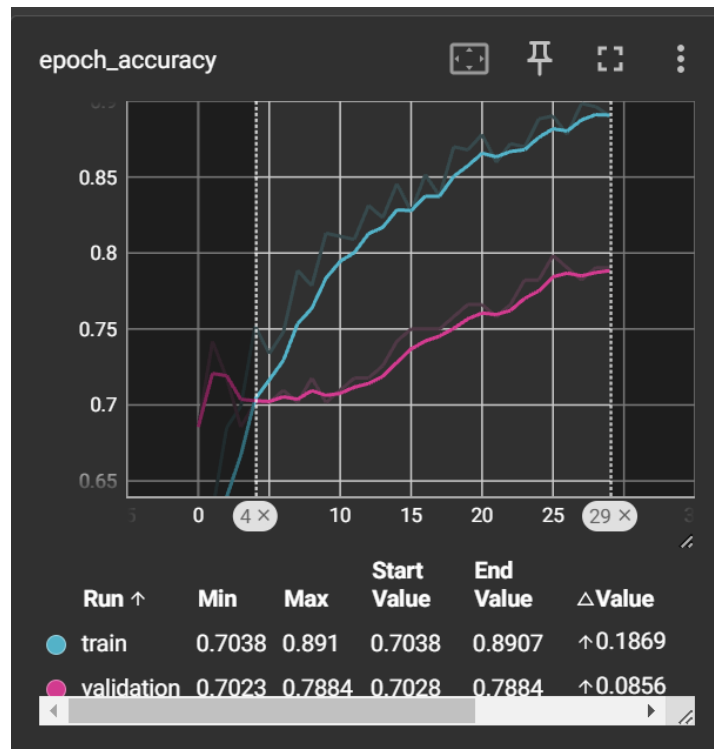
```
Best Model Performance on Test Data:
Best Validation Accuracy: 0.7903

ROC AUC Score: 0.7840
Precision: 0.8142
Recall: 0.8762
F1 Score: 0.8440
```

As we look at the confusion matrix, there are many more true negatives than before. There is still a large number of false positives which is not what we would like to see.



We take one final look at learning curves. We are still intersecting early with validation still have a significant gap from our training. This is indicative of overfitting that will need to be addressed.



Conclusion

Keras and TensorFlow have very powerful tools that are useful in machine learning tasks. The library is extensive but provides tools that make development streamlined. The TensorBoard really can make the difference when analyzing model performance amongst other things. The setup took a little getting used to, but eventually was worth it. Keras Tuner really improved the model's performance.

The final model still has large rooms for improvement. The end results vary from run to run with some runs being as low as 72% and others slightly over 80% which, I believe, can be improved through feature engineering and model improvement.

Overall, this project has been a good hands-on experience utilizing Keras and TensorFlow. I have gained much more experience with MLP's. Even with simple neural networks it is clear the power they can possess in machine learning tasks.